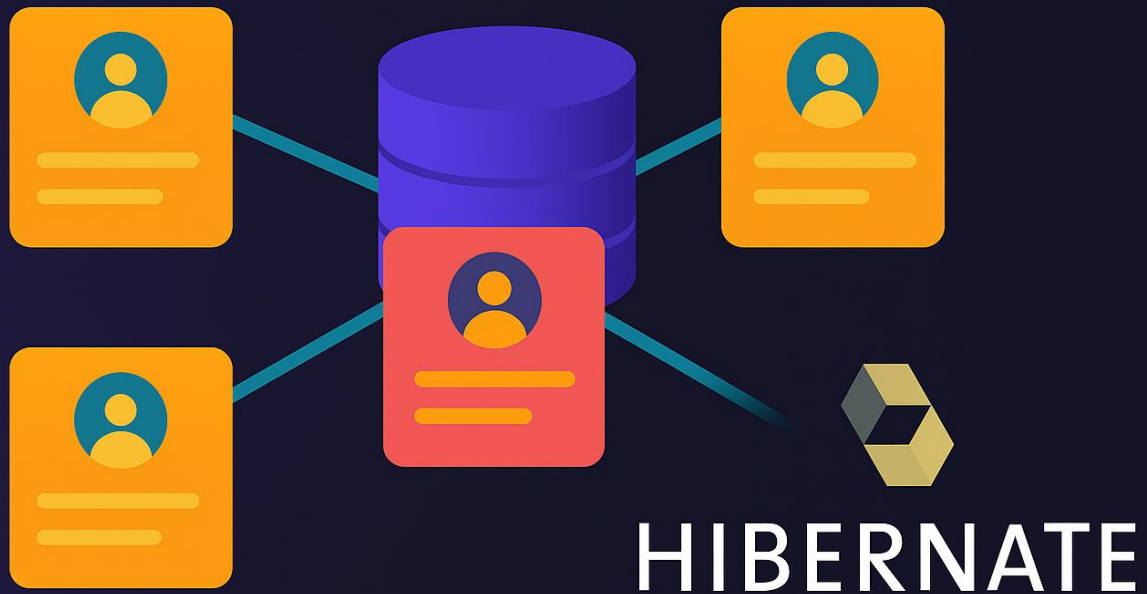


ASOCIACIONES CON HIBERNATE



🔧 Laboratorio: Creación de DAOs y clases de prueba en Hibernate + JPA

En esta lección vamos a resolver el laboratorio propuesto, en el cual crearemos las clases DAO (Data Access Object) para cada una de nuestras entidades. También prepararemos métodos de acceso como `listar`, `insertar`, `actualizar`, `eliminar` y `buscar por ID`.

📁 1. Clase Base: `GenericDAO.java`

✂ Ruta: `src/main/java/sga/dao/GenericDAO.java`

Se crea una clase abstracta que centraliza la configuración del `EntityManagerFactory` y del `EntityManager`. Esta clase será extendida por los demás DAOs.

Código Final:

```
package sga.dao;
```

```
import jakarta.persistence.*;

public abstract class GenericDAO implements AutoCloseable {

    protected final EntityManagerFactory emf;
    protected final EntityManager em;

    private static final String PU = "HibernateJpaPU";

    public GenericDAO() {
        this.emf = Persistence.createEntityManagerFactory(PU);
        this.em = emf.createEntityManager();
    }

    protected EntityManager getEntityManager() {
        return this.em;
    }

    @Override
    public void close() {
        if (em.isOpen()) {
            em.close();
        }
        if (emf.isOpen()) {
            emf.close();
        }
    }
}
```



2. DAO de Domicilio: DomicilioDAO.java



Ruta: src/main/java/sga/dao/DomicilioDAO.java

Esta clase extiende de `GenericDAO` e implementa operaciones básicas para la entidad `Domicilio`.

Código Final:

```
package sga.dao;

import jakarta.persistence.*;
import sga.domain.Domicilio;

import java.util.List;
```

```
public class DomicilioDAO extends GenericDAO {

    public List<Domicilio> listar() {
        String jpql = "SELECT d FROM Domicilio d";
        return getEntityManager().createQuery(jpql, Domicilio.class).getResultList();
    }

    public Domicilio buscarPorId(Domicilio domicilio) {
        return getEntityManager().find(Domicilio.class, domicilio.getIdDomicilio());
    }

    public void insertar(Domicilio domicilio) {
        var tx = em.getTransaction();
        try {
            tx.begin();
            em.persist(domicilio);
            tx.commit();
        } catch (Exception e) {
            if (tx.isActive()) {
                tx.rollback();
            }
            throw new PersistenceException("Error al insertar domicilio", e);
        }
    }

    public void modificar(Domicilio domicilio) {
        var tx = em.getTransaction();
        try {
            tx.begin();
            em.merge(domicilio);
            tx.commit();
        } catch (Exception e) {
            if (tx.isActive()) {
                tx.rollback();
            }
            throw new PersistenceException("Error al modificar domicilio", e);
        }
    }

    public void eliminar(Domicilio domicilio) {
        var tx = em.getTransaction();
        try {
            tx.begin();
            em.remove(em.merge(domicilio));
        }
```

```
        tx.commit();
    } catch (Exception e) {
        if (tx.isActive()) {
            tx.rollback();
        }
        throw new PersistenceException("Error al eliminar domicilio", e);
    }
}

@Override
public void close() {
    super.close(); // Llama al cierre definido en GenericDAO
}
}
```

✓ Conclusión

Ya creamos correctamente:

- La clase base `GenericDAO` que centraliza la configuración de conexión.
- La clase `DomicilioDAO` con operaciones CRUD + `buscarPorId`.

💡 En la siguiente lección seguiremos creando las demás clases DAO (`ContactoDAO`, `AlumnoDAO`, `CursoDAO`, `AsignacionDAO`), todas siguiendo la misma estructura base.

Sigue adelante con tu aprendizaje 🚀, ¡el esfuerzo vale la pena!

¡Saludos! 🙌

Ing. Marcela Gamiño e Ing. Ubaldo Acosta

Fundadores de [GlobalMentoring.com.mx](https://www.globalmentoring.com.mx)